

Practice Question Set

1. What is an array?
2. What is another name for an array variable?
3. Why do we need arrays in programming instead of using regular individual variables?
4. Write the declaration syntax for a one-dimensional (1-D) array and define its parts.
5. What operator is used to access array elements, and what are its operands?
6. What are the starting and ending index rules for an array of size N in C?
7. Can you leave the size brackets empty during a standalone 1-D array declaration? Explain.
8. What is a 2-D array, and what structural interface does it map data into?
9. What is the syntax specification for a 2-D array, and how do you calculate its total element capacity?
10. What is a three-dimensional (3-D) array, and what real-world analogy describes it?

Important Interview/Viva Questions

11. Explain what happens when a local array is left uninitialized. What is this trap called?
12. What is the difference between "Partial Initialization" and "Implicit Sizing" in 1-D arrays?

13. What is "Array Overflow," and what does it cause during compilation?
14. When declaring and initializing a 2-D array simultaneously, which dimension is strictly mandatory and which is optional?
15. Why does a 2-D array require nested loops to print or navigate its structure? What does each loop control?
16. Give the structural purpose and access analogy for 1-D, 2-D, and 3-D arrays.
17. If you declare `int arr[][3] = {12, 34, 56, 78, 91, 23};`, how many rows will the compiler automatically allocate?
18. What does the term "contiguous memory location" mean for an array's physical layout?
19. Given `int a[2][3][2];`, how many nested loops are required to iterate over it completely, and what does each loop drive through?
20. Identify which of these declarations will throw a compilation error and why:

```
int a[];
```

```
int arr[2][];
```

```
int a[] = {1, 2, 3};
```

Answers :

1. What is an array?

Answer: An Array is a linear collection of elements of the same data type stored in contiguous (continuous) memory locations.

2. What is another name for an array variable?

Answer: An array is also known as a subscript variable.

3. Why do we need arrays in programming instead of using regular individual variables?

Answer: When managing large amounts of data (like 100 numbers), the regular method forces you to declare separate variables, making code lengthy, confusing, and difficult to maintain. The array method allows you to declare a single variable name with a fixed size where all elements are easily accessed using an index number.

4. Write the declaration syntax for a one-dimensional (1-D) array and define its parts.

Answer:

Syntax: data_type array_name [size];

data_type: The type of data elements stored inside (e.g., int, float, char).

array_name: Any valid C identifier name.

size: A positive natural number in square brackets specifying the total element capacity.

5. What operator is used to access array elements, and what are its operands?

Answer: The square brackets [] act as the subscript operator. Given an expression like a[3], the array name (a) and the index value (3) function as its operands.

6. What are the starting and ending index rules for an array of size N in C?

Answer: In C, array indexing always starts at 0 and ends at size - 1. For example, if size is 100, indices run from 0 to 99.

7. Can you leave the size brackets empty during a standalone 1-D array declaration? Explain.

Answer: No. You cannot leave the size brackets empty during a standalone declaration (e.g., int a[];). Doing so will result in a Compilation Error because the compiler needs to know how much memory to allocate.

8. What is a 2-D array, and what structural interface does it map data into?

Answer: A Two-Dimensional Array is an "Array of Arrays". It structurally maps data into a grid interface consisting of rows and columns, commonly referred to as a Matrix.

9. What is the syntax specification for a 2-D array, and how do you calculate its total element capacity?

Answer:

Syntax: data_type array_name [r] [c]; (where r is horizontal rows and c is vertical columns).

Total Elements Capacity: Calculated using the formula rows* columns (r * c).

10. What is a three-dimensional (3-D) array, and what real-world analogy describes it?

Answer: A 3-D array is an "Array of 2-D Arrays" (or an array of arrays of arrays). It can be conceptualized like a multi-page book, where each page contains an independent 2-D grid matrix.

Important Interview Answers :

11. Explain what happens when a local array is left uninitialized. What is this trap called?

Answer: When a local array declared inside a function is left uninitialized, its element blocks do not default to zero; instead, they contain completely unpredictable values. This is called **The Garbage Value Trap**.

12. What is the difference between "Partial Initialization" and "Implicit Sizing" in 1-D arrays?

Answer:

Partial Initialization: You define the size explicitly but provide fewer values. The compiler automatically populates all remaining slots with 0 (e.g., int a[5] = {10, 50}; results in [10, 50, 0, 0, 0]).

Implicit Sizing: You leave the size brackets completely empty but initialize the items concurrently on the same line. The compiler automatically calculates the exact size based on the provided element count.

13. What is "Array Overflow," and what does it cause during compilation?

Answer: Array Overflow happens when you attempt to initialize an array with more data values than its explicitly declared size allows (e.g., int a[5] = {10, 50, 30, 70, 20, 80, 40};). This results in a **Compilation Error**.

14. When declaring and initializing a 2-D array simultaneously, which dimension is strictly mandatory and which is optional?

Answer: Specifying the second dimension (**columns**) is strictly mandatory, while the first dimension (**rows**) remains optional because the compiler can deduce the rows based on the total element count initialized.

15. Why does a 2-D array require nested loops to print or navigate its structure? What does each loop control?

Answer: Because data is organized across two distinct geometric dimensions (rows and columns).

The **Outer loop** controls movement across horizontal rows.

The **Inner loop** controls movement across individual vertical columns within the current row.

16. Give the structural purpose and access analogy for 1-D, 2-D, and 3-D arrays.

Answer:

Array Dimension	Access Analogy	Structural Purpose
1-D Array	A single row of post office boxes	Linear list tracking
2-D Array	A flat spreadsheet page grid	Mathematical Matrix computing
3-D Array	A multi-page collection of spreadsheets	Complex spatial data tracking

17. If you declare `int arr[][3] = {12, 34, 56, 78, 91, 23};`, how many rows will the compiler automatically allocate?

Answer: The compiler will automatically determine that it needs **2 rows**. Since there are 3 mandatory columns, 6 elements divided by 3 columns equals 2 rows.

18. What does the term "contiguous memory location" mean for an array's physical layout?

Answer: It means that no matter how large the array is, it always consumes memory in a single continuous, sequential block without any gaps or spatial separation between individual elements.

19. Given `int a[2][3][2];`, how many nested loops are required to iterate over it completely, and what does each loop drive through?

Answer: It requires **three nested loops**.

Loop 1 drives through the Page Layers.

Loop 2 drives through the Rows on the active page.

Loop 3 drives through the Columns in the active row.

20. Identify which of these declarations will throw a compilation error and why:

Answer:

`int a[];` **Throws an error** because standalone declarations cannot have empty size brackets without concurrent initialization values.

`int arr[2][];` **Throws an error** because omitting the second dimension (columns) of a 2-D array is strictly forbidden.

`int a[] = {1, 2, 3};` **Valid (No Error)** because it uses implicit sizing; the compiler correctly computes a size of 3 based on the initialized values.